

You are a senior Angular and Supabase full-stack architect.

Help me design and implement a production-ready company profile website with an administration dashboard and a secure client report portal.

Developer background

I am primarily an Angular frontend developer and have limited backend experience. Explain backend and security decisions clearly, but generate production-quality code rather than simplified demonstration code.

Technology stack

Use:

- Angular with standalone components
- TypeScript strict mode
- Angular Reactive Forms
- Angular Router
- Angular Material or a clean reusable component system
- Supabase PostgreSQL
- Supabase Authentication
- Supabase Storage
- Supabase Row-Level Security
- Cloudflare Pages for Angular hosting
- Supabase migrations for database changes

Use a scalable feature-based Angular architecture.

Application purpose

The application is a company profile website with two areas:

1. A public company website
2. A protected report portal and admin dashboard

Administrators upload company reports. Authorized users can log in and access only the reports assigned to them or their role.

Reports may contain confidential company information.

Public pages

Create the following public pages:

- Home

- About Us
- Services
- Reports landing page
- Contact Us
- Login
- Privacy Policy
- Not Found

The public website must be responsive and SEO-friendly.

User roles

Implement the following roles:

1. Super Admin
2. Full access
3. Manage users
4. Manage roles
5. Manage report categories
6. Manage reports
7. Assign report permissions
8. View audit logs
9. Report Admin
10. Create, update, publish and delete reports
11. Manage report categories
12. Cannot manage Super Admin users
13. Client User
14. View and download only reports that are assigned directly to the user or assigned to one of the user's roles

Required modules

Authentication

Implement:

- Login
- Logout

- Forgot password
- Reset password
- Email verification
- Current-user session handling
- Protected routes
- Role-based route guards
- Inactive-user handling

Do not rely only on Angular route guards for security. Enforce authorization using Supabase Row-Level Security.

Report management

Each report should include:

- ID
- Title
- Description
- Category
- Report year
- Report date
- Original filename
- Private storage path
- File size
- MIME type
- Public/private indicator
- Draft/published/archived status
- Uploaded by
- Created date
- Updated date

Administrators must be able to:

- Add a report
- Upload a report file
- Update report information
- Replace an existing file
- Delete or archive a report
- Publish and unpublish a report
- Filter reports
- Assign a report to users or roles

Users must be able to:

- View permitted reports
- Search reports
- Filter by category, year and date
- View report details

- Download permitted reports

User management

Administrators must be able to:

- Invite a user
- View users
- Update a user profile
- Activate or deactivate a user
- Assign roles
- View the reports or roles assigned to a user

Do not allow normal administrators to create or modify a Super Admin unless the current user is a Super Admin.

Categories

Administrators must be able to create, update, activate and deactivate report categories.

Audit logging

Record important actions such as:

- User login
- Report upload
- Report update
- Report deletion
- Report download
- Permission assignment
- User creation
- User deactivation

Database tables

Design migrations for:

- profiles
- roles
- user_roles
- report_categories
- reports
- report_permissions
- audit_logs

Use UUID primary keys where appropriate.

Add foreign keys, indexes, timestamps and constraints.

Avoid duplicating authentication passwords or sensitive Supabase Auth information in the profiles table.

Report permission rules

A user may access a report when at least one of these conditions is true:

- The report is published and marked public
- The report is assigned directly to the user
- The report is assigned to one of the user's roles
- The user is a Super Admin
- The user is an authorized Report Admin

A deactivated user must not access private reports.

Storage security

Use a private Supabase Storage bucket named `reports`.

Never store confidential reports in a public bucket.

Do not save permanent public file URLs.

Generate short-lived signed URLs only after validating that the current user can access the report.

Validate:

- Allowed extensions
- MIME types
- Maximum file size
- Empty files
- Duplicate uploads
- Unsafe filenames

Store files using generated paths such as:

```
reports/{year}/{report-id}/{generated-file-name}
```

Do not trust the original filename as a storage path.

Angular architecture

Propose a feature-based structure similar to:

- core
- auth
- guards

- interceptors
- services
- models
- shared
- components
- directives
- pipes
- validators
- features
- public-site
- authentication
- reports
- users
- categories
- audit-logs
- admin-dashboard

Use typed interfaces and avoid `any`.

Use reusable loading, empty-state, confirmation-dialog, error-message and file-upload components.

State handling

Use Angular signals and services for application state.

Do not introduce a large state-management library unless there is a clear requirement.

Error handling

Implement:

- Friendly user-facing error messages
- Centralized application error handling
- Loading states
- Empty states
- Upload progress
- Retry handling where appropriate
- Logging without exposing sensitive information

User experience

The UI must support:

- Desktop, tablet and mobile
- Accessible labels
- Keyboard navigation

- Confirmation before destructive actions
- Pagination
- Search
- Filters
- Clear validation messages
- Responsive tables or cards

Deliverables

Produce the project in the following order:

1. Clarified assumptions and scope
2. System architecture
3. Database entity relationship explanation
4. SQL migrations
5. Row-Level Security policies
6. Storage policies
7. Angular folder structure
8. Authentication implementation
9. Report CRUD implementation
10. Private upload and signed-download implementation
11. User and role management
12. Admin dashboard
13. Audit logging
14. Unit and integration testing
15. Environment configuration
16. Deployment instructions
17. Production security checklist
18. Handover documentation

For every phase:

- Explain the objective
- List the files to create or modify
- Provide complete code
- Explain how to test it
- Do not skip security policies
- Do not place service-role keys in the Angular application
- Stop after completing the current phase so I can review it before continuing